

MT5 Optimizer Studio - User & Deployment Guide

Analyze MetaTrader 5 optimization results, surface the **robust** parameter sets (not curve-fit), refine your search to cut future test time, check **prop-firm** suitability, and track live accounts - in one dashboard.

1. What it does

Capability	Summary
Import	Ingests MT5 Strategy Tester "Optimization Results -> Export to XML" (streamed; handles 45 MB / 15k+ passes)
Rank	Scores every pass 0-100 for robustness (weighted, per-run-normalized)
Prop-firm	Flags passes that fit a firm's max-drawdown limit
Refine	Emits a narrowed <code>.set</code> - pins dead inputs, tightens live ones - to shrink the next optimization
Portfolio	Tracks live accounts via Myfxbook (equity, drawdown, recovery factor)
Link	Pins a backtested set to the live account running it (backtest vs live)

2. Run it locally

```
cd C:\Users\dazha\Projects\mt5-optimizer-studio
.venv\Scripts\python -m uvicorn app.main:app --port 8137
```

Open <http://localhost:8137>. First run creates `mt5studio.db` (SQLite) in the project folder. Locally there is **no login** - the gate only turns on when `APP_PASSWORD` is set (see Deployment).

3. The workflow

- Import a run.** MT5 -> Strategy Tester -> Optimization Results -> right-click -> *Export to XML* (enable **Forward** testing first). In the app's **Optimization run** panel, choose the XML, enter the **EA name**, click **Import & Analyze**. Large files import in the background - the panel shows progress.
- Rank for robustness.** The leaderboard shows the top 20 passes by Robustness Score. Adjust the weight sliders (Profit Factor / Drawdown / Profit / Consistency) and hit **Re-score**. The profitability gate and min-trades filter drop curve-fit / low-sample junk automatically.
- Prop-firm check.** Turn on "evaluate for prop firm", set **Max total DD %** (8-10%). The **Prop** column flags [pass] / [fail] (hover [fail] for the reason); "show prop-safe only" filters to fundable sets.
- Refine.** In the **Refine** panel, upload the base `.set` template you optimized -> **Refine & Download**. You get a drop-in `.set` with a far smaller search space for the next MT5 optimization.
- Track live accounts.** In **Portfolio** -> **Connect a Myfxbook account**, enter your Myfxbook email/password -> **Sync**. Accounts appear with equity, current & max drawdown, and recovery factor; click one for its equity curve. Use the "link a set" dropdown to pin a leaderboard pass to the account running it.

4. Metrics cheat-sheet

- Robustness Score (0-100)** - the app's headline ranking.
- Consistency** - forward result ÷ back result; ~1.0 = held up out of sample.
- Max DD** - worst peak-to-valley ever; the prop-firm killer / your risk history.

- **Cur DD** - current floating drawdown (balance -> equity).
- **Recovery Factor** - profit ÷ max DD; **>3** excellent, **1-3** solid, **<0** net loss.

5. Deploy to the cloud (Render)

Gets you a public **HTTPS link**. The app is FastAPI + Postgres; the `render.yaml` blueprint provisions both.

5.1 Prerequisites

- A **GitHub** account (to host the code)
- A **Render** account - <https://render.com> (free tier works to start)

5.2 Push the code to GitHub

From the project folder:

```
git init
git add .
git commit -m "mt5 optimizer studio"
git branch -M main
git remote add origin https://github.com/<you>/mt5-optimizer-studio.git
git push -u origin main
```

`.gitignore` already excludes `mt5studio.db`, `.venv`, and `.env`, so your local data and secrets are never pushed.

5.3 Deploy on Render

1. Render dashboard -> **New -> Blueprint**.
2. Connect the GitHub repo. Render reads `render.yaml` and proposes a **web service** + a **Postgres database**.
3. Click **Apply**. Render builds and provisions the database (`DATABASE_URL` is injected automatically).

5.4 Set your secrets

In the web service -> **Environment**, set:

Variable	Value	Purpose
<code>APP_PASSWORD</code>	your chosen password	turns the login gate ON
<code>SECRET_KEY</code>	auto-generated by the blueprint	signs session cookies
<code>SECURE_COOKIES</code>	<code>true</code> (preset)	HTTPS-only cookies
<code>MT5_AGENT_TOKEN</code>	a random secret (optional)	for the VPS agent
<code>DATABASE_URL</code>	injected from the DB	Postgres connection

Save -> Render redeploys. Visit the service URL -> you'll get the **login page**. Sign in with `APP_PASSWORD`.

5.5 First use on the cloud

The cloud database starts **empty** - import a run from the dashboard (your local data is separate). Connect Myfxbook from the Portfolio panel as usual.

5.6 Updating the app

```
git add . && git commit -m "update" && git push
```

Render auto-deploys on push.

5.7 Custom domain (optional)

Render service -> **Settings** -> **Custom Domains** -> add your domain and follow the DNS instructions. HTTPS is issued automatically.

6. Live MT5 tracking from your VPS (optional)

For real-time equity beyond Myfxbook's daily data, run the agent **on the VPS where MT5 lives** - it attaches to the running terminal (no password needed) and pushes snapshots to your hosted app.

```
pip install MetaTraders5 httpx
set STUDIO_URL=https://<your-render-url>
set MT5_AGENT_TOKEN=<same value you set on Render>
python -m scripts.mt5_agent --interval 60
```

Read-only by design - it never trades.

7. Security notes

- The app is **gated by APP_PASSWORD** on the cloud; keep it strong.
- **Myfxbook credentials are never stored** - they're used once per sync and discarded.
- All traffic is **HTTPS** on Render; session cookies are HTTPS-only.
- Never commit `.env` files or the database (already git-ignored).

8. Troubleshooting

- **Login loop / can't stay signed in** - ensure `SECRET_KEY` is set and stable.
- **Import seems stuck** - large files parse in the background; the panel polls and shows `processing` then `done`. Check the run appears in the dropdown.
- **Myfxbook "Invalid session"** - resolved in-app (session token is URL-decoded). Set `MYFXBOOK_DEBUG=1` in the server env for response-level logs.
- **DB errors after a schema change** - this MVP creates tables on startup; managed migrations (Alembic) come with the multi-tenant phase.

9. Roadmap

- **Phase 2 - multi-tenant client accounts**: per-user login and private workspaces (each client sees only their own runs and accounts).
- Optional: periodic auto-sync, conditional-dependency refiner, plateau scoring, full-report ingestion for daily-DD / consistency prop checks.

10. Command line (optional)

```
# Summarize & import a results.xml
./venv/Scripts/python -m scripts.import_optxml summarize path/to/results.xml

./venv/Scripts/python -m scripts.import_optxml persist path/to/results.xml "Archangel X" --symbol XAGUSD --tr M

# Score a run
./venv/Scripts/python -m scripts.score_run 1 --pr 0.2 --dd 0.3 --profile 0.3 --cons

# Refine a run into a narrowed set file
./venv/Scripts/python -m scripts.refine_run 1 base base-set out-refined-set

# Tests
./venv/Scripts/python -m pytest
```